

MATLAB Source Code: Regula Falsi Method

(False Position Method)

```
% Clearing Screen
clc

% Setting x as symbolic variable
syms x;

% Input Section
y = input('Enter non-linear equations: ');
a = input('Enter first guess: ');
b = input('Enter second guess: ');
e = input('Tolerable error: ');

% Finding Functional Value
fa = eval(subs(y,x,a));
fb = eval(subs(y,x,b));

% Implementing Bisection Method
if fa*fb > 0
    disp('Given initial values do not bracket the root. ');
else
    c = a - (a-b) * fa/(fa-fb);
    fc = eval(subs(y,x,c));
    fprintf('\n\n a\t\t b\t\t c\t\t f(c)\n');
    while abs(fc)>e
        fprintf('%f\t%f\t%f\t%f\n',a,b,c,fc);
```

```

        if fa*fc < 0
            b = c;
            fb = eval(subs(y,x,b));
        else
            a = c;
            fa = eval(subs(y,x,a));
        end
        c = a - (a-b) * fa / (fa-fb);
        fc = eval(subs(y,x,c));
    end
    fprintf('\nRoot is: %f\n', c);
end

```

Regula Falsi MATLAB Output

Enter non-linear equations: $\sin(x) + \cos(x) + \exp(x) - 8$
Enter first guess: 2
Enter second guess: 3
Tolerable error: 0.00001

a	b	c	f(c)
2.000000	3.000000	2.010374	-0.054516
2.010374	3.000000	2.015152	-0.025119
2.015152	3.000000	2.017349	-0.011551
2.017349	3.000000	2.018358	-0.005306
2.018358	3.000000	2.018821	-0.002437
2.018821	3.000000	2.019034	-0.001119
2.019034	3.000000	2.019132	-0.000514
2.019132	3.000000	2.019177	-0.000236
2.019177	3.000000	2.019197	-0.000108
2.019197	3.000000	2.019207	-0.000050
2.019207	3.000000	2.019211	-0.000023
2.019211	3.000000	2.019213	-0.000010

Root is: 2.019214

Python Source Code: False Position (Regula Falsi) Method

```
# Defining Function
def f(x):
    return x**3-5*x-9

# Implementing False Position Method
def falsePosition(x0,x1,e):
    step = 1
    print('\n\n*** FALSE POSITION METHOD IMPLEMENTATION ***')
    condition = True
    while condition:
        x2 = x0 - (x1-x0) * f(x0) / ( f(x1) - f(x0) )
        print('Iteration-%d, x2 = %0.6f and f(x2) = %0.6f' % (step, x2, f(x2)))

        if f(x0) * f(x2) < 0:
            x1 = x2
        else:
            x0 = x2

        step = step + 1
        condition = abs(f(x2)) > e

    print('\nRequired root is: %0.8f' % x2)

# Input Section
```

```

x0 = input('First Guess: ')
x1 = input('Second Guess: ')
e = input('Tolerable Error: ')

# Converting input to float
x0 = float(x0)
x1 = float(x1)
e = float(e)

#Note: You can combine above two section like this
# x0 = float(input('First Guess: '))
# x1 = float(input('Second Guess: '))
# e = float(input('Tolerable Error: '))

# Checking Correctness of initial guess values and
false positioning
if f(x0) * f(x1) > 0.0:
    print('Given guess values do not bracket the
root.')
    print('Try Again with different guess values.')
else:
    falsePosition(x0,x1,e)

```

False Position Program Output

```

First Guess: 2
Second Guess: 3
Tolerable Error: 0.00001

```

```

*** FALSE POSITION METHOD IMPLEMENTATION ***
Iteration-1, x2 = 2.785714 and f(x2) = -1.310860

```

Iteration-2, $x_2 = 2.850875$ and $f(x_2) = -0.083923$

Iteration-3, $x_2 = 2.854933$ and $f(x_2) = -0.005125$

Iteration-4, $x_2 = 2.855180$ and $f(x_2) = -0.000312$

Iteration-5, $x_2 = 2.855196$ and $f(x_2) = -0.000019$

Iteration-6, $x_2 = 2.855196$ and $f(x_2) = -0.000001$

Required root is: 2.85519648